Chaînage avant (Forward Chaining)

Le chaînage avant (Forward Chaining)

1. Transition : analogie → chaînage avant

Nous avons étudié le raisonnement par analogie, qui consiste à utiliser une situation passée pour proposer une solution à un problème similaire.

Le **chaînage avant**, quant à lui, permet de déduire automatiquement de nouvelles informations à partir de faits connus et de règles logiques.

2. Définition

Le chaînage avant est une méthode d'inférence qui part d'une **base de faits (BF)** et applique les règles d'une **base de règles (BR)** pour produire de nouvelles conclusions.

Chaque conclusion déduite est ajoutée à la base de faits.

Le processus continue jusqu'à ce qu'aucun nouveau fait ne puisse être déduit.

Principe :

Faits connus → Application des règles → Nouveaux faits → Répétition

3. Fonctionnement

Supposons :

- Base de faits initiale : BF = {X}
- Règle R1 : X → Y
- Règle R2 : Y → Z

Étape 1 : X est connu. On applique R1 et on déduit Y.

BF = {X, Y}

Étape 2 : Y est maintenant connu. On applique R2 et on déduit Z.

BF = {X, Y, Z}

Étape 3 : Aucune règle ne permet de produire un nouveau fait. Le raisonnement s'arrête.

À retenir : les conclusions déduites deviennent de nouveaux faits qui peuvent déclencher d'autres règles.

4. Exercice — Diagnostic informatique

Énoncé :

Un ordinateur ne démarre pas. Un test confirme que sa batterie est vide.

Base de faits :

- ORNDP : l'ordinateur ne démarre pas.
- TBV : la batterie est vide.

Base de règles :

- R1 : TBV → MDN (manque d'énergie).
- R2 : MDN → BRCH (il faut brancher le chargeur).

Questions :

1. Déterminer les conclusions obtenues par chaînage avant.
2. Représenter les faits et les règles en Python avec SymPy.
3. Programmer l'application automatique des règles.
4. Afficher la base de faits finale.

Résolution logique :

BF initiale = {ORNDP, TBV}

Application de R1 → ajout de MDN.

Application de R2 → ajout de BRCH.

BF finale = {ORNDP, TBV, MDN, BRCH}

5. Implémentation Python avec SymPy

```
from sympy import symbols, Implies

ORNDP, TBV, MDN, BRCH = symbols("ORNDP TBV MDN BRCH")

# Base de faits
```

```
faits = {ORNDP, TBV}

# Base de règles
regle1 = Implies(TBV, MDN)
regle2 = Implies(MDN, BRCH)

regles = [regle1, regle2]

# Application des règles
for regle in regles:
    condition, conclusion = regle.args

    if condition in faits:
        faits.add(conclusion)

print(faits)
```

Explication :

- `for` parcourt les règles.
- `regle.args` récupère la condition et la conclusion.
- `condition in faits` vérifie si la règle peut être appliquée.
- `faits.add(conclusion)` ajoute la conclusion à la base de faits.

Dans notre exemple, ce code trouve toutes les conclusions parce que les règles sont dans le bon ordre.

6. Limite d'une simple boucle `for`

Une boucle `for` parcourt les règles **une seule fois**.

Supposons :

BF = {X}

Règles dans cet ordre :

- R1 : Y → Z
- R2 : X → Y

Premier parcours :

1. R1 : Y est absent → impossible de déduire Z.
2. R2 : X est présent → on ajoute Y.
3. La boucle se termine.

Résultat : BF = {X, Y}

Problème : Z n'a pas été déduit, alors que Y est maintenant connu.

Il faut donc recommencer le parcours des règles.

7. Solution : utiliser `while` et `for`

```
faits = {ORNDP, TBV}
regles = [regle2, regle1]  # Ordre inversé

changement = True

while changement:
    changement = False

    for regle in regles:
        condition, conclusion = regle.args

        if condition in faits and conclusion not in faits:
            faits.add(conclusion)
            changement = True

print(faits)
```

Rôle des instructions :

- `while changement` : répète le parcours tant qu'un nouveau fait a été découvert.
- `changement = False` : suppose qu'aucune nouvelle conclusion ne sera trouvée pendant ce parcours.
- `conclusion not in faits` : évite de considérer un fait déjà connu comme une nouveauté.
- `changement = True` : indique qu'un nouveau fait a été ajouté et qu'un autre parcours est nécessaire.

Évolution de la base de faits :

Passage	Nouveau fait	Base de faits
Initial	—	{ORNDP, TBV}
1	MDN	{ORNDP, TBV, MDN}
2	BRCH	{ORNDP, TBV, MDN, BRCH}
3	Aucun	Arrêt

Cette répétition permet de trouver toutes les conclusions déductibles avec nos règles simples, indépendamment de leur ordre.

8. Conclusion

Le chaînage avant est un raisonnement **dirigé par les données** : il commence par les faits connus et cherche les conclusions possibles.

Idée essentielle :

Si X est connu et que X → Y est une règle, alors Y devient un nouveau fait.

Le processus se répète jusqu'à ce qu'aucune nouvelle conclusion ne soit possible. On atteint alors un **point fixe**.

Cette méthode est notamment utilisée dans les systèmes experts et les moteurs de règles.

Remarque : notre programme traite des règles simples avec une seule condition positive. Des règles plus complexes nécessitent une implémentation adaptée.